

Implementação do SGBD para Gestão de Supermercado

Leonardo Kenji

Tiago Egrid

Julia Ayumi Watanabe

4º semestre 28/11/2025

1. Introdução e Contexto do Projeto

Este projeto visa resgatar e concluir a implementação de um sistema de gestão essencial para um supermercado. A consultoria de TI anterior entregou apenas o Modelo Lógico do Banco de Dados, deixando o trabalho incompleto. O supermercado enfrenta uma necessidade urgente de um sistema funcional que gerencie de forma integrada suas operações críticas: vendas, controle de estoques, compras e recursos humanos (funcionários). Além disso, a gerência exige relatórios claros e acionáveis para fundamentar a tomada de decisão.

O escopo deste trabalho consiste em transformar o modelo lógico existente em um Sistema de Gerenciamento de Banco de Dados (SGBD) relacional e funcional. Após a implementação da estrutura (SQL), o banco de dados foi populado com dados sintéticos que simulam o fluxo operacional real. A etapa final e crucial envolveu a criação de ferramentas de análise de dados, utilizando a linguagem Python e suas bibliotecas (Pandas, Matplotlib, Seaborn), para gerar relatórios conectados e visualizações interativas.

2. Problemáticas e Justificativa da Solução

2.1. Problemáticas Atuais do Supermercado

A ausência de um SGBD devidamente implementado e funcional gera ineficiências operacionais significativas, que impactam diretamente a rentabilidade do negócio:

- **Risco de Ruptura e Excesso de Estoque:** A falta de um controle de estoque preciso impede o rastreamento do fluxo de produtos. Isso resulta em perda de vendas por falta de produto ou em excesso de estoque (capital parado e risco de perdas por validade).
- **Gestão de Vendas Deficiente:** Sem dados estruturados, a gerência não consegue identificar os produtos de maior giro, avaliar o desempenho individual da equipe de vendas ou reconhecer tendências sazonais para otimizar o planejamento de receita.
- **Tomada de Decisão:** Decisões estratégicas sobre precificação, promoções, volume de compras e contratações são baseadas em intuição, e não em dados concretos. Essa abordagem aumenta o risco de prejuízos e impede o crescimento sustentável.

2.2. Objetivo do Projeto

O objetivo central deste projeto é implementar um SGBD relacional robusto, fornecendo à gerência as ferramentas necessárias para uma gestão baseada em dados. Os resultados esperados são:

- **Garantia de Integridade e Segurança dos Dados:** Assegurar que todos os dados operacionais sejam armazenados de forma íntegra, consistente e sem redundância.
- **Otimização de Estoque e Compras:** Fornecer alertas automáticos de estoque crítico e relatórios de giro, permitindo que a equipe de compras otimize o capital de giro e previna a perda de vendas.
- **Decisões Estratégicas:** Gerar Relatórios (Views SQL) e Visualizações Interativas (Python) que transformam dados brutos em inteligência de negócios, capacitando a gerência a tomar decisões fundamentadas sobre preço, logística, mix de produtos e gestão de pessoal.

3. Modelo Lógico Implementado e Premissas

O projeto foi desenvolvido com base estrita no modelo lógico fornecido pela consultoria anterior. A implementação seguiu o princípio da integridade referencial, utilizando instruções SQL DDL para criar a estrutura do banco de dados relacional.

3.1. Estrutura do Modelo Lógico (Entidades)

O modelo abrange as áreas centrais do negócio:

- Categoria_Produto (catp_id, catp_nome)
- Produto (prod_id, prod_nome, prod_preco, prod_qtd_estoque, catp_catp_id)
- Fornecedor (forn_id, forn_nome, forn_cnpj, forn_telefone)
- Produto_Fornecedor (pf_prod_id, pf_forn_id)
- Cliente (cli_id, cli_nome, cli_cpf, cli_telefone)
- Funcionario (func_id, func_nome, func_cargo, func_salario, func_data_admissao)
- Endereco_Cliente (endc_id, endc_rua, endc_numero, endc_bairro, endc_cidade, endc_estado, endc_cli_id)
- Venda (venda_id, venda_data, venda_valor_total, venda_cli_id, venda_func_id)
- Item_Venda (itemv_id, itemv_qtd, itemv_preco_unit, itemv_venda_id, itemv_prod_id)
- Compra (comp_id, comp_data, comp_valor_total, comp_forn_id, comp_func_id)
- Item_Compra (itemc_id, itemc_qtd, itemc_preco_unit, itemc_comp_id,, itemc_prod_id)
- Setor (setor_id, setor_nome)
- Funcionario_Setor (fs_func_id, fs_setor_id, fs_data_inicio, fs_data_fim)
- Pagamento (pag_id, pag_metodo, pag_valor, pag_data, pag_venda_id)
- Promocao (promo_id, promo_percentual, promo_inicio, promo_fim, promo_prod_id)

3.2. Implementação do Modelo (Código SQL)

As instruções CREATE TABLE foram utilizadas para criar a estrutura do banco de dados, estabelecendo as chaves primárias e todas as chaves estrangeiras para garantir a integridade referencial.

```
CREATE TABLE Categoria_Produto(
```

```
catp_id INTEGER PRIMARY KEY AUTOINCREMENT,  
catp_nome TEXT NOT NULL  
);
```

```
CREATE TABLE Produto(  
    prod_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    prod_nome TEXT NOT NULL,  
    prod_preco REAL NOT NULL,  
    prod_qtd_estoque INTEGER NOT NULL,  
    catp_catp_id INTEGER,  
    FOREIGN KEY(catp_catp_id) REFERENCES Categoria_Produto(catp_id)  
);
```

```
CREATE TABLE Fornecedor(  
    forn_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    forn_nome TEXT NOT NULL,  
    forn_cnpj TEXT,  
    forn_telefone TEXT  
);
```

```
CREATE TABLE Produto_Fornecedor(  
    pf_prod_id INTEGER,  
    pf_forn_id INTEGER,  
    PRIMARY KEY(pf_prod_id, pf_forn_id),  
    FOREIGN KEY(pf_prod_id) REFERENCES Produto(prod_id),  
    FOREIGN KEY(pf_forn_id) REFERENCES Fornecedor(forn_id)  
);
```

```
CREATE TABLE Cliente(  
    cli_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
cli_nome TEXT NOT NULL,  
cli_cpf TEXT,  
cli_telefone TEXT  
);
```

```
CREATE TABLE Funcionario(  
func_id INTEGER PRIMARY KEY AUTOINCREMENT,  
func_nome TEXT NOT NULL,  
func_cargo TEXT,  
func_salario REAL,  
func_data_admissao DATE  
);
```

```
CREATE TABLE Endereco_Cliente(  
endc_id INTEGER PRIMARY KEY AUTOINCREMENT,  
endc_rua TEXT,  
endc_numero TEXT,  
endc_bairro TEXT,  
endc_cidade TEXT,  
endc_estado TEXT,  
endc_cli_id INTEGER,  
FOREIGN KEY(endc_cli_id) REFERENCES Cliente(cli_id)  
);
```

```
CREATE TABLE Venda(  
venda_id INTEGER PRIMARY KEY AUTOINCREMENT,  
venda_data DATE,  
venda_valor_total REAL,  
venda_cli_id INTEGER,  
venda_func_id INTEGER,
```

```
FOREIGN KEY(venda_cli_id) REFERENCES Cliente(cli_id),  
FOREIGN KEY(venda_func_id) REFERENCES Funcionario(func_id)  
);
```

```
CREATE TABLE Item_Venda(  
    itemv_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    itemv_qtd INTEGER,  
    itemv_preco_unit REAL,  
    itemv_venda_id INTEGER,  
    itemv_prod_id INTEGER,  
    FOREIGN KEY(itemv_venda_id) REFERENCES Venda(venda_id),  
    FOREIGN KEY(itemv_prod_id) REFERENCES Produto(prod_id)  
);
```

```
CREATE TABLE Compra(  
    comp_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    comp_data DATE,  
    comp_valor_total REAL,  
    comp_forn_id INTEGER,  
    comp_func_id INTEGER,  
    FOREIGN KEY(comp_forn_id) REFERENCES Fornecedor(forn_id),  
    FOREIGN KEY(comp_func_id) REFERENCES Funcionario(func_id)  
);
```

```
CREATE TABLE Item_Compra(  
    itemc_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    itemc_qtd INTEGER,  
    itemc_preco_unit REAL,  
    itemc_comp_id INTEGER,  
    itemc_prod_id INTEGER,
```

```
FOREIGN KEY(itemc_comp_id) REFERENCES Compra(comp_id),  
FOREIGN KEY(itemc_prod_id) REFERENCES Produto(prod_id)  
);
```

```
CREATE TABLE Setor(  
    setor_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    setor_nome TEXT  
);
```

```
CREATE TABLE Funcionario_Setor(  
    fs_func_id INTEGER,  
    fs_setor_id INTEGER,  
    fs_data_inicio DATE,  
    fs_data_fim DATE,  
    PRIMARY KEY(fs_func_id, fs_setor_id, fs_data_inicio),  
    FOREIGN KEY(fs_func_id) REFERENCES Funcionario(func_id),  
    FOREIGN KEY(fs_setor_id) REFERENCES Setor(setor_id)  
);
```

```
CREATE TABLE Pagamento(  
    pag_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    pag_metodo TEXT,  
    pag_valor REAL,  
    pag_data DATE,  
    pag_venda_id INTEGER,  
    FOREIGN KEY(pag_venda_id) REFERENCES Venda(venda_id)  
);
```

```
CREATE TABLE Promocao(  
    promo_id INTEGER PRIMARY KEY AUTOINCREMENT,
```



```
promo_percentual REAL,  
promo_inicio DATE,  
promo_fim DATE,  
promo_prod_id INTEGER,  
FOREIGN KEY(promo_prod_id) REFERENCES Produto(prod_id)  
);
```

3.3. População do Banco de Dados (DML)

Para validar as regras de negócio e a performance das *queries*, o banco foi populado com dados sintéticos verossímeis, simulando fluxos reais de entrada (compras com fornecedores) e saída (vendas ao consumidor final).

- Categorias (ex: Laticínios, Hortifrúti).
- Produtos com diferentes níveis de estoque.
- Funcionários e Clientes.
- Registros de Compras, Vendas e Itens de Venda/Compra para simular a movimentação.

4. Análise Visual e Interativa com Python (Relatórios)

As Views SQL atuam como relatórios pré-definidos, unindo dados de múltiplas tabelas.

Utilizando as bibliotecas Python pandas, matplotlib e seaborn, foram desenvolvidos 7 relatórios visuais para transformar os dados das views em gráficos claros e informativos. Cada relatório foca em uma área críticas de gestão:

Análise de Demanda: A view `vw_produtos_mais_vendidos` identifica os principais produtos do faturamento. A análise revelou, por exemplo, que itens básicos como "Pão Francês" lideram em volume, enquanto itens de maior valor agregado impactam a receita total.

Sazonalidade e Receita: Através da `vw_desempenho_vendas_mensal`, monitoramos a saúde financeira, permitindo comparar o crescimento mês a mês .

Gestão de Ativos: A view `vw_estoque_por_categoria` expõe a alocação de capital. Identificamos que a categoria "Laticínios" representa quase 48% do volume estocado, exigindo atenção redobrada quanto à validade dos produtos.

Performance de RH: O relatório `vw_vendas_detalhadas` introduz a meritocracia baseada em dados, quantificando a contribuição individual de cada funcionário para o faturamento.

Sistema de Alerta Antecipado: Implementamos a `vw_estoque_critico` como uma ferramenta proativa. Ela categoriza produtos em níveis "BAIXO" (≤ 50) e "CRÍTICO" (≤ 30), automatizando a necessidade de recompra e prevenindo a perda de vendas.

Lucratividade e Fornecedores: As análises de Margem de Lucro e Desempenho de Fornecedores (`vw_custo_medio...` e `vw_desempenho_fornecedores`) fecham o ciclo, permitindo que a gerência negocie melhor com parceiros estratégicos e foque em produtos com maior *spread* entre custo e venda.

Relatório 1: Produtos Mais Vendidos

vw_produtos_mais_vendidos (Foco em Vendas e Demanda)

Objetivo: Identificar os produtos que mais contribuem para o volume e o faturamento do supermercado.

```
CREATE VIEW vw_produtos_mais_vendidos AS

SELECT

p .prod_nome AS produto,

cp .catp_nome AS categoria,

SUM ( iv .itemv_qtd) AS total_qtd_vendida,

SUM ( iv .itemv_qtd * iv .itemv_preco_unit) AS valor_total_vendido

FROM Item_Venda iv

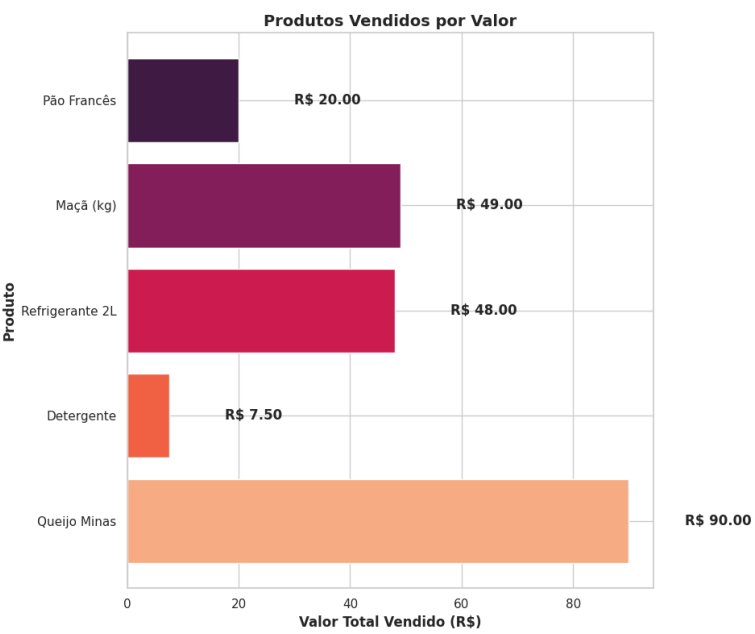
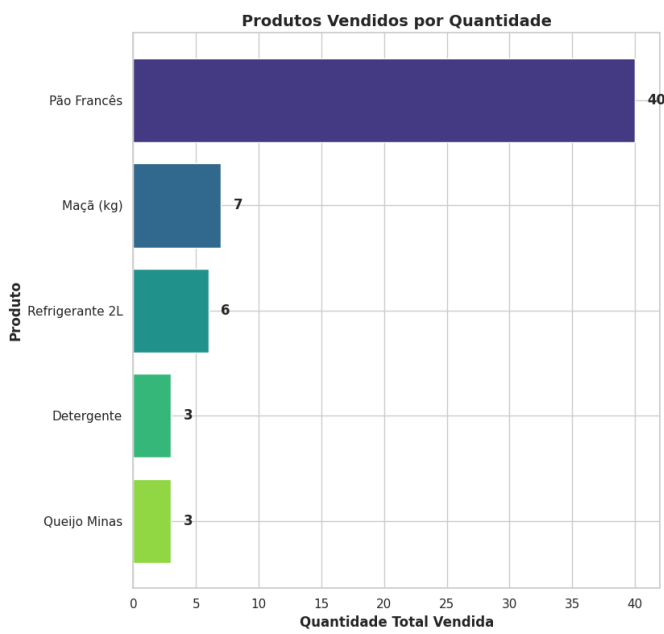
JOIN Produto p ON iv .itemv_prod_id = p .prod_id

JOIN Categoria_Produto cp ON p .catp_catp_id = cp .catp_id

GROUP BY p .prod_nome, cp .catp_nome

ORDER BY total_qtd_vendida DESC

LIMIT 5;
```



Relatório 2: Desempenho de Vendas Mensal

vw_desempenho_vendas_mensal (Foco em Desempenho Financeiro)

Objetivo: Acompanhar a evolução da receita e do número de vendas ao longo do tempo.

Esta view rastreia a Receita Mensal e o total de vendas. É a principal métrica para avaliar o crescimento ou declínio do negócio ao longo do tempo, auxiliando no planejamento financeiro e na definição de metas.

```
CREATE VIEW vw_desempenho_vendas_mensal AS
```

```
SELECT
```

```
    strftime('%Y-%m', venda_data) AS ano_mes,
```

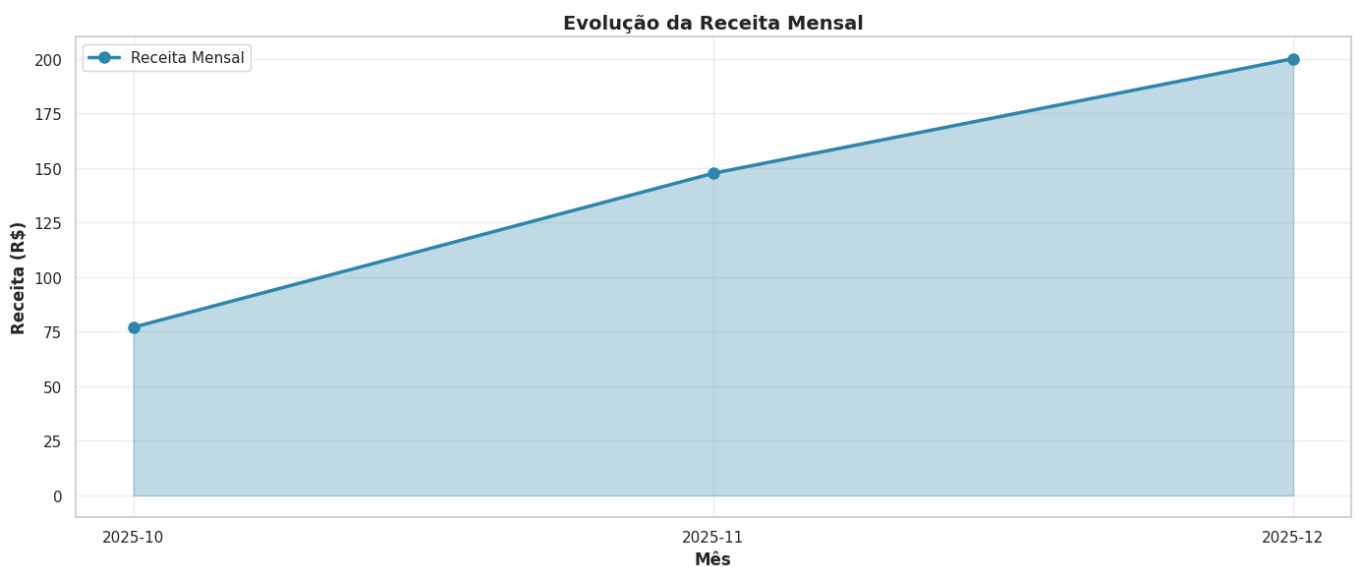
```
    SUM(venda_valor_total) AS receita_mensal,
```

```
    COUNT(venda_id) AS total_vendas
```

```
FROM Venda
```

```
GROUP BY ano_mes
```

```
ORDER BY ano_mes DESC;
```



Relatório 3: Análise de Estoque por Categoria

vw_estoque_por_categoria (Foco em Estoque e Capital)

Objetivo: Visualizar a composição do estoque e o valor médio dos produtos em cada categoria.

Esta view oferece uma visão agregada, mostrando a Quantidade Total de Estoque e o Preço Médio por categoria de produto. Isso é vital para analisar onde o capital do

supermercado está alocado e identificar as categorias que demandam maior volume de armazenamento.

```
CREATE VIEW vw_estoque_por_categoria AS
```

```
SELECT
```

```
    cp.catp_nome AS categoria,
```

```
    SUM(p.prod_qtd_estoque) AS qtd_total_estoque,
```

```
    AVG(p.prod_preco) AS preco_medio_produto
```

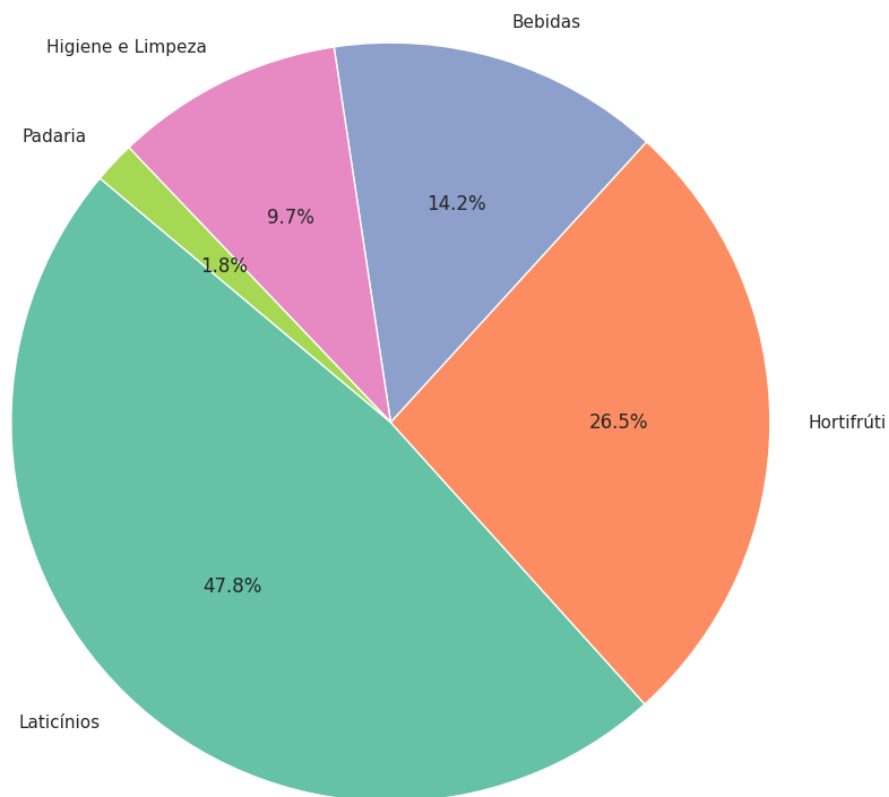
```
FROM Produto p
```

```
JOIN Categoria_Produto cp ON p.catp_catp_id = cp.catp_id
```

```
GROUP BY cp.catp_nome
```

```
ORDER BY qtd_total_estoque DESC;
```

Distribuição da Quantidade Total de Estoque por Categoria



A alta concentração em perecíveis (Laticínios) sugere alto risco de desperdício se o giro não for rápido.

Relatório 4: Desempenho dos Funcionários

vw_vendas_detalhadas (Foco em Desempenho e capacitação)

Objetivo: Medir a performance da equipe de vendas com base no volume e no valor das vendas realizadas.

Com essa view a gerência pode reconhecer e recompensar os funcionários de melhor desempenho. Além disso, pode identificar oportunidades de treinamento para a equipe, focando em técnicas para aumentar o ticket médio das vendas.

CREATE VIEW vw_vendas_detalhadas **AS**

SELECT

v.venda_id,

v.venda_data,

c.cli_nome **AS** cliente,

f.func_nome **AS** funcionario,

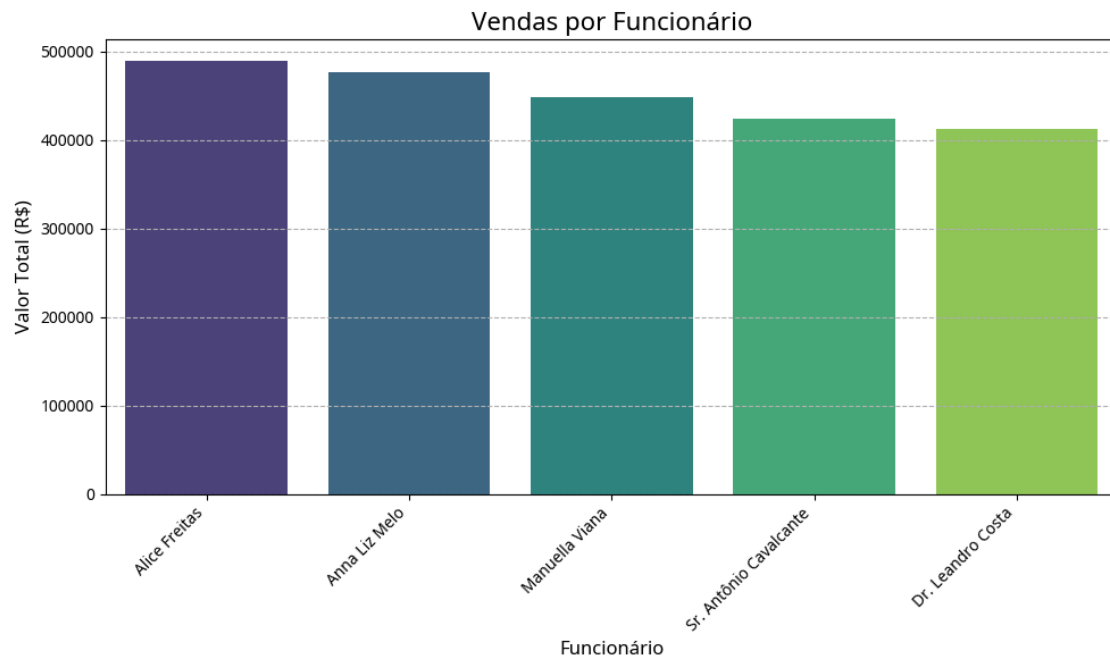
v.venda_valor_total

FROM Venda v

JOIN Cliente c **ON** v.venda_cli_id = c.cli_id

JOIN Funcionario f **ON** v.venda_func_id = f.func_id

ORDER BY v.venda_data **DESC** ;



Relatório 5: Alertas de Gestão

vw_estoque_critico (Estoque Crítico e Pagamentos)

Objetivo: esta View atua como um **sistema de alerta** para a equipe de compras.

- Ela lista apenas os produtos que possuem estoque abaixo ou igual a **50 unidades**.
- A coluna nivel_alerta classifica o estado do produto:
 - **CRÍTICO**: Estoque de 30 unidades ou menos.
 - **BAIXO**: Estoque entre 31 e 50 unidades.

O objetivo principal é prevenir a **ruptura de estoque** e a consequente perda de vendas, garantindo que a equipe de compras possa agir rapidamente.

CREATE VIEW vw_estoque_critico **AS**

SELECT

p.prod_nome,

p.prod_qtd_estoque,

CASE

WHEN *p.prod_qtd_estoque* <= 30 **THEN** 'CRÍTICO'

```

    WHEN p.prod_qtd_estoque <= 50 THEN 'BAIXO'

    ELSE 'NORMAL'

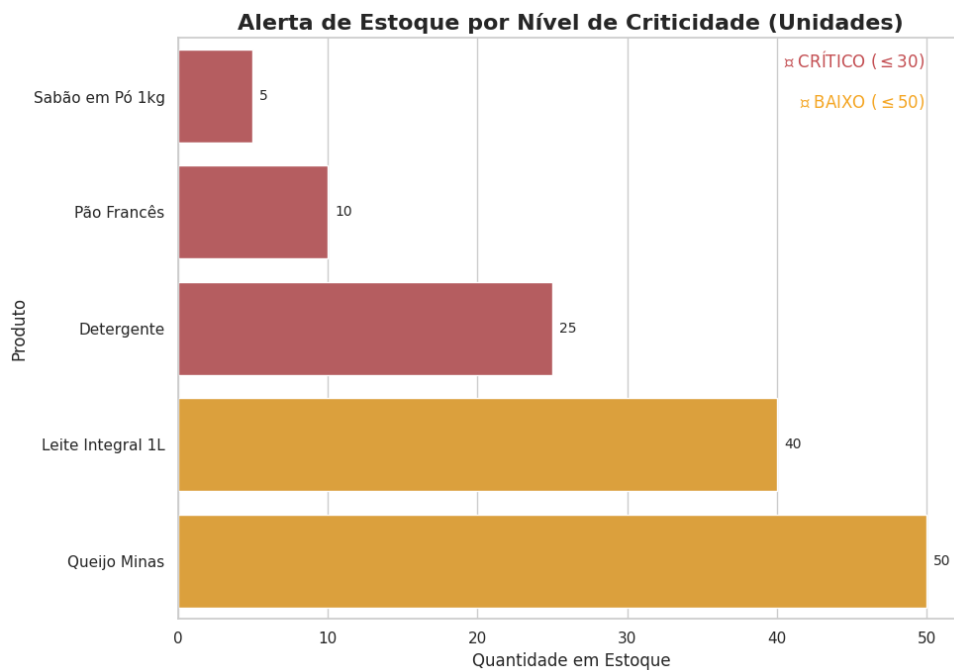
END AS nivel_alerta

FROM Produto p

WHERE p.prod_qtd_estoque <= 50

ORDER BY p.prod_qtd_estoque ASC;

```



Relatório 6: Análise de Margem de Lucro

Objetivo: Identificar os produtos mais lucrativos, comparando o custo de compra com o preço de venda.

A gerência pode criar promoções "leve junto" com produtos de alta margem para impulsionar a lucratividade geral. Também pode renegociar preços com fornecedores de produtos cuja margem está abaixo do esperado.

```

CREATE VIEW vw_custo_medio_por_produto AS

SELECT

    p.prod_nome AS produto,

    cp.catp_nome AS categoria,

```



```

AVG(ic.itemc_preco_unit) AS custo_medio_unitario,

p.prod_preco AS preco_venda,

(p.prod_preco - AVG(ic.itemc_preco_unit)) AS margem_lucro_unitaria,

ROUND(((p.prod_preco - AVG(ic.itemc_preco_unit)) / p.prod_preco * 100), 2) AS
percentual_margem

FROM Item_Compra ic

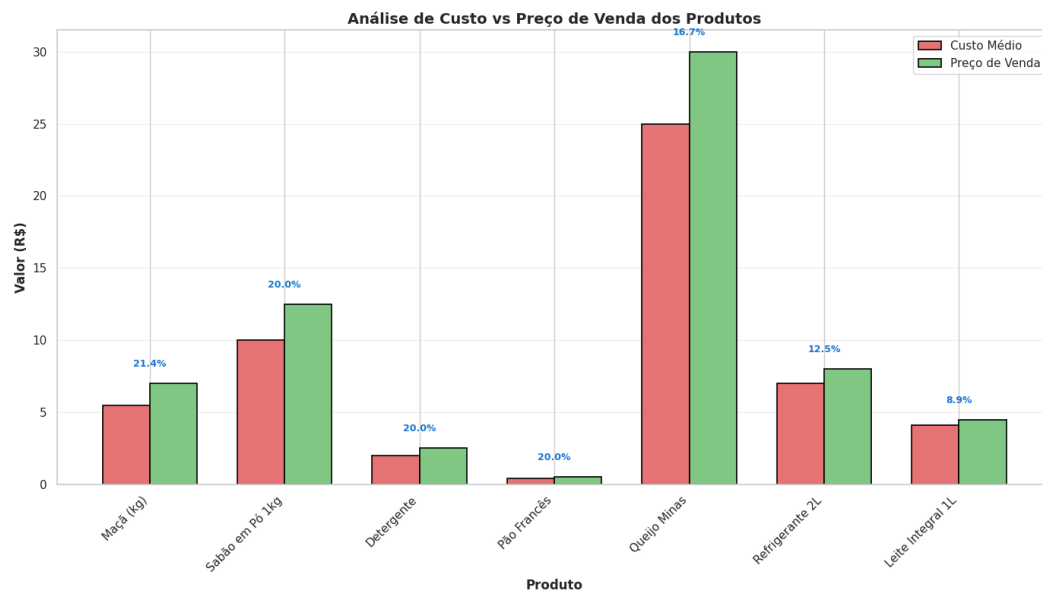
JOIN Produto p ON ic.itemc_prod_id = p.prod_id

JOIN Categoria_Produto cp ON p.catp_catp_id = cp.catp_id

GROUP BY p.prod_nome, cp.catp_nome, p.prod_preco

ORDER BY percentual_margem DESC;

```



Relatório 7: Análise de Desempenho de Fornecedores

Objetivo: Permite à gerência identificar os **fornecedores mais estratégicos** (maior volume de compras) para negociações de preço, e os **menos utilizados**, onde pode haver oportunidade de diversificação.

```

CREATE VIEW vw_desempenho_fornecedores AS

```

```

SELECT

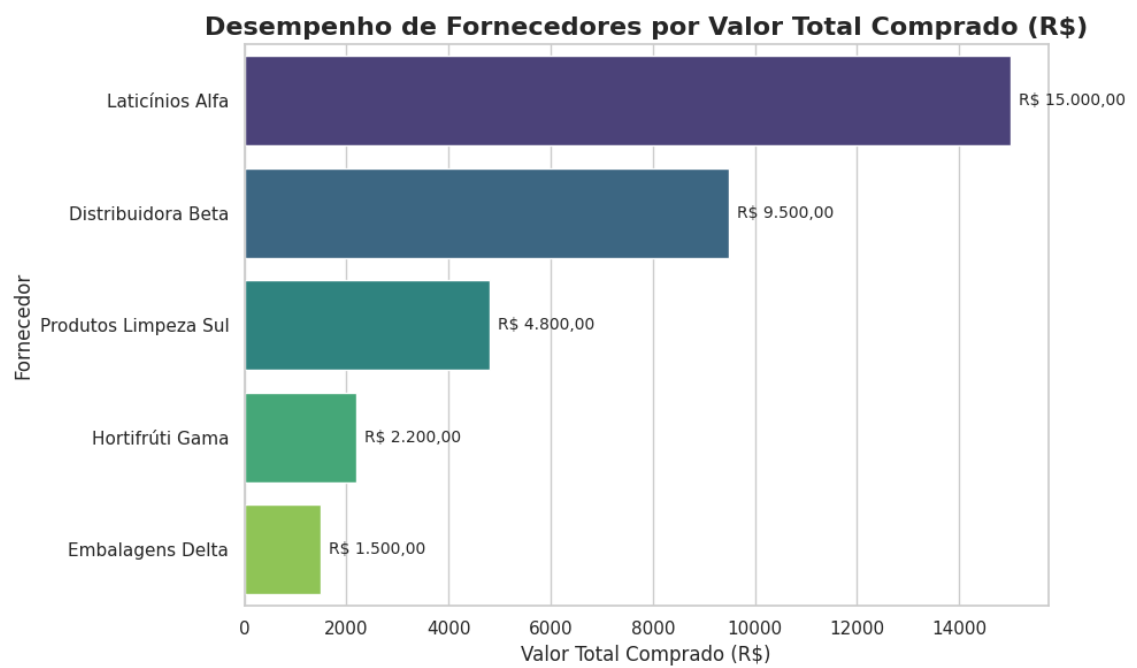
```

```

    f.forn_nome AS fornecedor,

```

```
COUNT(c.comp_id) AS total_compras,  
SUM(c.comp_valor_total) AS valor_total_comprado  
FROM Fornecedor f  
JOIN Compra c ON f.forn_id = c.comp_forn_id  
GROUP BY f.forn_nome  
ORDER BY valor_total_comprado DESC;
```



5. Conclusão e Próximos Passos

A implementação do SGBD do supermercado, a partir do modelo lógico fornecido, resultou em uma ferramenta robusta e estruturada, capaz de:

1. Organizar e Integrar dados de forma segura (Compras, Vendas, Estoque e RH).
2. Fornecer Alertas Críticos (Estoque Crítico).
3. Gerar Análises Visuais (Python) que transformam dados brutos em inteligência de negócios (Desempenho Mensal e Performance da Equipe).

As ferramentas desenvolvidas fornecem uma visão 360º do negócio, desde a performance de um único produto até a saúde financeira mensal da empresa. Com os relatórios de estoque crítico, análise de margem de lucro e desempenho de funcionários, o supermercado está agora equipado para otimizar suas operações, aumentar a rentabilidade e garantir um crescimento sustentável no longo prazo.

Manutenção e Escalabilidade: O design relacional rigoroso e o uso adequado de chaves primárias e estrangeiras garantem não apenas a integridade dos dados no presente, mas também a alta manutenibilidade do sistema. A estrutura permite uma fácil escalabilidade futura, possibilitando a adição de novos módulos essenciais ao crescimento do negócio, como um sistema de fidelidade do cliente ou a integração com um sistema de ponto de venda (PDV), sem comprometer a arquitetura relacional atual. Com os relatórios de estoque crítico, análise de margem de lucro e desempenho de funcionários, o supermercado está agora equipado para otimizar suas operações, aumentar a rentabilidade e garantir um crescimento sustentável no longo prazo.